

Assertions and Testing Functions

Day 4 Afternoon - Debugging, Exceptions, and Testing

30 min teaching + 30 min exercises

```
def test_normalize_status():  
    raw_status = " ACTIVE "  
  
    result =  
normalize_status(raw_status)  
  
    assert result == "active"
```

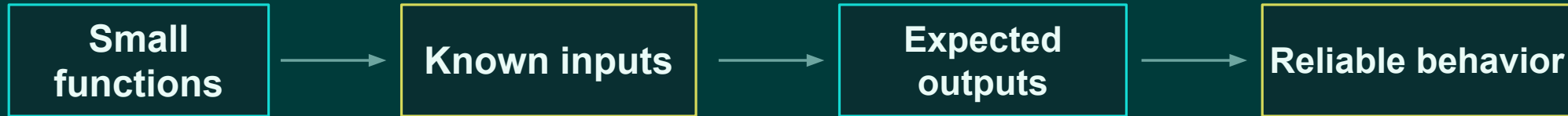
```
assert calculate_percentage(0, 100) ==  
0  
assert calculate_percentage(100, 100)  
== 100  
assert is_supported_file(" REPORT.CSV  
") is True
```

Goal: use known inputs to prove that small functions behave correctly.

BLOCK 31

Where this block fits

Block 30 made failure behavior intentional. Block 31 proves function behavior systematically.



Instruction focus

- why testing exists
- expected vs actual behavior
- assertions
- normal, boundary, invalid cases
- Arrange-Act-Assert
- regression tests

Practice focus

Students write assertions for the functions they created earlier in Day 4.

BLOCK 31

Why test?

Programs often fail at unusual values, not typical ones.

```
calculate_percentage(50, 100)
```

Manual check
Looks good: 50%

But what about these?

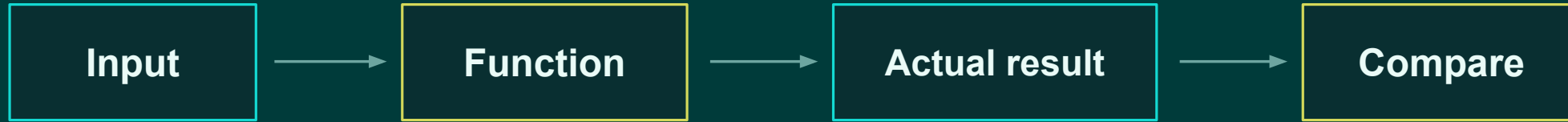
```
calculate_percentage(0, 100)  
calculate_percentage(100, 100)  
calculate_percentage(1, 3)  
calculate_percentage(50, 0)
```

Testing asks
Given a known input, does the
function produce the expected
behavior?

Testing turns “I think it works” into evidence.

Expected versus actual

A test compares what should happen with what did happen.



Example

Input: 50, 100
Expected: 50
Actual: 50
Result: pass

If they differ

The test gives evidence that either the implementation or the expectation needs attention.

BLOCK 31

Assertions

An assertion verifies that a condition is true.

```
result = calculate_percentage(50, 100)
assert result == 50
```

```
assert calculate_percentage(40, 100) ==
50
# AssertionError
```

If true
Execution continues.

If false
Python raises
AssertionError.

Mental model
assert means: “Stop if this
assumption is not true.”

BLOCK 31

A simple test function

A test can be saved as a reusable function.

```
def test_calculate_percentage():  
    result = calculate_percentage(50, 100)  
  
    assert result == 50
```

What this does

1. Calls the function
2. Stores the result
3. Verifies expected behavior

Naming pattern

Use names that explain the behavior being checked.

```
def test_percentage_at_half_completion():  
    assert calculate_percentage(50, 100) == 50
```

BLOCK 31

Arrange, Act, Assert

This pattern keeps tests readable.

Arrange
Prepare the input.

Act
Run the code.

Assert
Verify the result.

```
def test_normalize_status():  
  
    # Arrange  
    raw_status = " ACTIVE "  
  
    # Act  
    result = normalize_status(raw_status)  
  
    # Assert  
    assert result == "active"
```

BLOCK 31

Test normal, boundary, and invalid cases

One happy-path test is not enough.

Normal
Typical values

50 of 100

Boundary
Edges of allowed range

0 and 100

Invalid
Values outside the rules

-1 and 101

```
assert is_valid_percentage(50) is True
assert is_valid_percentage(0) is True
assert is_valid_percentage(100) is True
assert is_valid_percentage(-1) is False
assert is_valid_percentage(101) is False
```


BLOCK 31

Testing Boolean functions

Boolean results should be checked explicitly.

```
def is_supported_file(file_name):  
    file_name = file_name.strip().lower()  
  
    return (  
        file_name.endswith(".csv")  
        or file_name.endswith(".json")  
    )
```

```
def test_supported_csv():  
    assert is_supported_file("records.csv")  
    is True  
  
def test_unsupported_txt():  
    assert is_supported_file("records.txt")  
    is False
```

Why use is True / is False here?
It makes the expectation visually obvious for beginners.

BLOCK 31

Testing normalization

Small cleaning functions are excellent testing targets.

```
def normalize_status(status):  
    return status.strip().lower()
```

Cases to prove
lowercase
uppercase
leading whitespace
trailing whitespace

```
assert normalize_status("active") == "active"  
assert normalize_status("ACTIVE") == "active"  
assert normalize_status(" active") == "active"  
assert normalize_status("active ") == "active"
```

If the same rule matters in production, it deserves a test.

BLOCK 31

Regression tests

When you fix a bug, add a test that would have caught it.

Bug report

“REPORT.CSV ” was rejected even though CSV files should be accepted.



Fix

Strip whitespace and lowercase the file name before checking the extension.

```
def test_csv_with_spaces_and_capitals():  
    assert is_supported_file("REPORT.CSV ") is True
```

Purpose

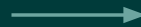
If a future change accidentally reintroduces the bug, this test fails immediately.

BLOCK 31

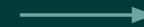
Testing functions independently

Small functions can be tested without running the whole program.

```
for record in records:  
    # 30 lines of processing  
    # normalization  
    # validation  
    # classification  
    # printing
```



Refactor
Break the logic into
named pieces.



```
assert normalize_status(" ACTIVE ") ==  
"active"  
assert classify_error_count(6) ==  
"rejected"
```

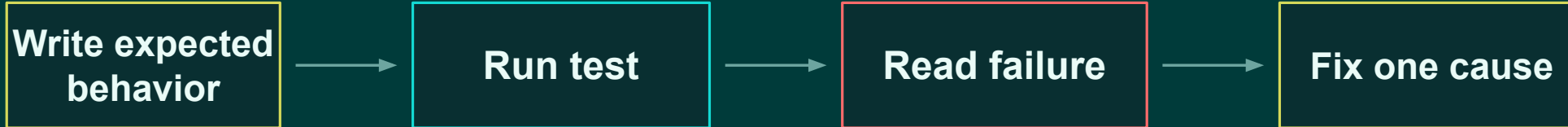
Design lesson

Code that is easier to test is usually easier to understand.

BLOCK 31

Instructor demo: test first, then inspect failures

Do not treat a failing test as a disaster. Treat it as evidence.



Demonstrate with
`classify_error_count()`

Boundary bug
5 errors should be warning, not
rejected.

The test tells you where reality differs from the rule.

BLOCK 31

Guided exercises: 30-minute plan

Students write assertions for functions developed during the morning.

0–6 min
`normalize_status()`

6–12 min
`is_supported_file()`

12–18 min
`calculate_percentage()`

18–24 min
`classify_error_count()`

24–30 min
challenge table

Instructor goal

Students should write expected behavior before changing code.

Normal. Boundary. Invalid.

BLOCK 31

Exercise 1: test normalize_status()

Prove that whitespace and capitalization are handled.

```
def normalize_status(status):  
    return status.strip().lower()
```

Task

Write four assertions: lowercase, uppercase, leading whitespace, trailing whitespace.

Starter

```
assert normalize_status("ACTIVE") ==  
"active"
```

6 minutes

Checkpoint

Can you explain exactly which behavior each assertion proves?

BLOCK 31

Exercise 2: test is_supported_file()

Cover supported, unsupported, uppercase, and whitespace cases.

```
def is_supported_file(file_name):  
    file_name = file_name.strip().lower()  
  
    return (  
        file_name.endswith(".csv")  
        or file_name.endswith(".json")  
    )
```

Required tests

CSV

JSON

TXT

Uppercase extension

Extra whitespace

6 minutes

The bug usually lives in the case you did not think to test.

BLOCK 31

Exercise 3: test calculate_percentage()

Include normal and boundary cases.

```
def calculate_percentage(completed, total):  
    return completed / total * 100
```

Required tests

0%
50%
100%

Think ahead

What should happen
when total is zero?
That belongs with
exception testing later.

```
assert calculate_percentage(0, 100) == 0  
assert calculate_percentage(50, 100) == 50  
assert calculate_percentage(100, 100) == 100
```

6 minutes

BLOCK 31

Exercise 4: test classify_error_count()

Boundary tests matter because category rules change at specific values.

```
def classify_error_count(error_count):  
    if error_count == 0:  
        return "valid"  
  
    if error_count <= 5:  
        return "warning"  
  
    return "rejected"
```

Required tests

0 → valid

1 → warning

5 → warning

6 → rejected

Why these?

They test the exact points where behavior changes.

6 minutes

BLOCK 31

Applied challenge: build the test table first

Define correct behavior before implementing the function.

Input	Expected
0 errors	valid
1 error	warning
5 errors	warning
6 errors	rejected
-1 errors	exception

Task

Create assertions for every row. Then implement or correct `classify_error_count()`.

Rule

A negative error count is invalid input, not a warning category.

Challenge

BLOCK 31

Applied challenge starter code

Students fill in assertions and then fix the function.

```
def classify_error_count(error_count):  
    if error_count < 0:  
        raise ValueError("error_count cannot be  
negative")  
  
    if error_count == 0:  
        return "valid"  
  
    if error_count <= 5:  
        return "warning"  
  
    return "rejected"
```

```
assert classify_error_count(0) == "valid"  
assert classify_error_count(1) == "warning"  
assert classify_error_count(5) == "warning"  
assert classify_error_count(6) == "rejected"  
  
# Run this and observe the exception:  
classify_error_count(-1)
```

Instructor note

pytest will handle expected exceptions more cleanly in the next block. For now, observe the behavior.

BLOCK 31

Block 31 wrap-up

Testing gives us a systematic way to demonstrate that code works.

Students can explain

- expected vs actual behavior
- assert
- test functions
- Arrange-Act-Assert
- normal, boundary, invalid cases
- regression tests

Students can build

- assertions for normal behavior
- boundary tests
- Boolean function tests
- normalization tests
- test tables before code

Big idea

Small functions are easier to test. Tested functions are easier to trust.

Next: automate those tests with pytest and refactor for testability.